

선행연구조사_0807

👤 작성자

한동훈(20반_7855)

▼ 1. 로컬 LLM 개념

1. 사용자의 개인 컴퓨터, 서버에 직접 설치하여 실행하는 생성형 AI 모델

a. 동작 방식

- i. 사용자 입력 (Prompt)
- ii. 로컬 LLM 실행 프로그램(Ollama, LM Studio..)
- iii. LLM 모델(Llama, Gemma, Mistral...)
- iv. 응답 (Response)

b. 클라우드 LLM과의 차이점

	클라우드 LLM	로컬 LLM
실행위치	제공자의 서버	사용자 PC
인터넷	필수	대부분 불필요
데이터처리	서버	로컬
개인정보	서버전송가능	외부 전송 없음
모델수정	불가능	가능
성능	매우 높음	PC성능에 따라 달라짐
포렌식 대상	웹브라우저, 클라우드데이터...	로컬파일, 로그 데이터베이스...

c. 장점

- i. 개인정보보호
- ii. 오프라인 사용
- iii. 높은 확장성 (원하는 모델을 자유롭게 다운받아 사용)
- iv. 다양한 오픈소스

d. 단점

- i. 높은 하드웨어 요구사항
- ii. 초기 설치 용량
- iii. 성능차이(GPU, CPU에 따름)
- iv. 관리필요(모델업데이트, 설정관리, 로그관리를 사용자가 직접 수행)

e. 디지털 포렌식 관점에서의 로컬 LLM

- 실행 과정에서 다양한 아티팩트를 생성 (모델파일, 설정파일, 캐시, 로그, 대화기록)
→ 로컬 저장장치에 기록 (분석 대상)

2. 사용자의 개인 컴퓨터, 서버에 직접 설치하여 실행하는 생성형 AI 모델

a. 동작 방식

- i. 사용자 입력 (Prompt)
- ii. 로컬 LLM 실행 프로그램(Ollama, LM Studio..)
- iii. LLM 모델(Llama, Gemma, Mistral...)
- iv. 응답 (Response)

b. 클라우드 LLM과의 차이점

	클라우드 LLM	로컬 LLM
실행위치	제공자의 서버	사용자 PC
인터넷	필수	대부분 불필요
데이터처리	서버	로컬
개인정보	서버전송가능	외부 전송 없음
모델수정	불가능	가능
성능	매우 높음	PC성능에 따라 달라짐
포렌식 대상	웹브라우저, 클라우드데이터...	로컬파일, 로그 데이터베이스...

c. 장점

- i. 개인정보보호
- ii. 오프라인 사용
- iii. 높은 확장성 (원하는 모델을 자유롭게 다운받아 사용)
- iv. 다양한 오픈소스

d. 단점

- i. 높은 하드웨어 요구사항
- ii. 초기 설치 용량
- iii. 성능차이(GPU, CPU에 따름)
- iv. 관리필요(모델업데이트, 설정관리, 로그관리를 사용자가 직접 수행)

e. 디지털 포렌식 관점에서의 로컬 LLM

- 실행 과정에서 다양한 아티팩트를 생성 (모델파일, 설정파일, 캐시, 로그, 대화기록)
→ 로컬 저장장치에 기록 (분석 대상)

▼ 2. 주요 로컬 LLM 프로그램

1. Ollama (<https://github.com/ollama/ollama/>)
 - a. 로컬환경에서 LLM을 손쉽게 실행, 관리할 수 있도록 개발된 오픈소스 플랫폼
 - b. 사용자는 명령어를 통해 모델을 다운로드, 실행 가능
 - c. 로컬에서 REST API를 제공해 다른 애플리케이션과의 연동 가능
2. LM Studio (<https://lmstudio.ai/docs/app>)
 - a. GUI 기반 인터페이스를 기반으로 로컬 LLM을 실행할 수 있는 프로그램
 - b. Hugging Face(AI 및 머신러닝 개발자의 오픈소스 ai 플랫폼) 모델 검색 및 다운로드
 - c. GGUF(LLM 모델의 도커) 호환, REST API 제공
 - d. 문서기반 질의응답 기능 지원
3. GPT4ALL (<https://docs.gpt4all.io/>)
 - a. GUI 기반 실행
 - b. LocalDocs 기능 지원
 - c. 오프라인 실행 가능
 - d. 다양한 공개 모델 지원
4. Jan (<https://www.jan.ai/docs/desktop>)
 - a. chatgpt와 유사한 GUI
 - b. 로컬모델 지원
 - c. OpenAI API 연동
 - d. 플러그인 지원
5. llama.cpp (<https://github.com/ggml-org/llama.cpp>)
 - a. c, c++기반
 - b. GGUF 모델 지원
 - c. CPU 최적화
 - d. GPU 가속
 - e. 다양한 공개 모델 실행

▼ 3. 기존 연구 조사

1. Forensic Implications of Localized AI: Artifact Analysis of Ollama, LM Studio, and llama.cpp (Shariq Murtuza) - 로컬 AI의 디지털 포렌식적 의의(Ollama, LM Studio, llama.cpp)

a. 연구 배경

- 로컬 LLM은 인터넷 연결 없이 동작하므로 사용자 활동을 외부에서 확인하기 어려움 → 악의적 사용자가 피싱메일, 악성코드, 정보분석에 활용할 경우 디지털 포렌식 조사에 어려움 발생
- 따라서 로컬 LLM 환경에서 생성되는 아티팩트를 분석해 향후 디지털 포렌식 조사에 활용 가능한 방법론 제시

b. 연구 결과

i. Ollama

- 클라이언트 - 서버 구조로 동작
- 설치 과정에서 모델 버전 정보를 저장하는 json 파일이 생성되어 지속적으로 보존
- ollama run(CLI)를 이용한 프롬프트는 기록에 남지만 API, GUI를 통한 입력은 CLI 기록에 저장되지 않음
- 일부 사용자 활동은 확인 가능, 모든 대화 기록을 복원하기에는 한계 존재

ii. LM Studio

- 가장 많은 디지털 아티팩트 생성
- JSON 형태(프롬프트, AI응답, 사용모델, 환경설정)로 저장
- 100%의 복구율
- 포렌식 분석에 용이함

iii. llama.cpp

- 일반적인 명령 실행은 셸 히스토리에 일부 기록이 남을 수 있으나 대화형 모드에서는 프롬프트 기록이 디스크에 저장되지 않음
- 0%의 복구율
- 사용자 활동 분석을 위해 실행 중 확보한 RAM 메모리 분석이 필수적

c. 링크

- <https://arxiv.org/abs/2603.23996>

2. LangurTrace: Forensic analysis of local LLM applications - 로컬 LLM 애플리케이션에 대한 포렌식 분석 (Sungjo Jeong, Sangjin Lee, Jungheum Park)

a. 연구 배경

- 기존 연구들은 ChatGPT같은 클라우드 기반 서비스에 의존되어 있으나, 이 논문은 사용자 PC에서 직접 구동하는 로컬 LLM 환경의 분석에 초점을 맞추어 진행

b. 연구 결과

- LLM환경을 3가지로 분류하고 각각을 분석
 - 백엔드 런타임(ollama) : 모델을 다운로드하고 실행하는 역할
 - 주요 아티팩트 : api 호출 로그, 모델 메니페스트 파일 등을 통해 사용자가 어떤 모델을 언제 다운로드하고 실행했는지 파악
 - 클라이언트 인터페이스 (chatbox) : 사용자가 텍스트를 입력하고 결과를 확인하는 화면
 - 주요 아티팩트 : config.json 파일에 대파 세션, 프롬프트, api키 등 저장, 사용자가 업로드하거나 생성한 파일은 Base64형태로 로컬 디스크에 기록됨.
 - 통합 플랫폼 (LM Studio, Msty, Jan, GPT4ALL) : 백엔드와 클라이언트가 하나로 합쳐진 형태
 - LM STUDIO : 모델을 삭제해도 모델 다운로드URL과 해시값이 남아있어 과거 이력 추적 가능
 - Msty : 사용자가 업로드한 파일, 모델 설정 로그가 앱 내 ui에서 삭제되더라도 로컬 저장소에 그대로 남아있어 복구 가능
 - Jan : 상세로그파일에 대화내용, 설정, api키 등 구체적으로 기록됨
 - GPT4ALL : 파일 내에 대화 내용 뿐 아니라 업로드한 파일의 원본 데이터 전체가 함께 임베딩되어 저장됨
- LangurTrace 도구 개발
 - 사용자가 앱 내부 ui에서 삭제버튼을 눌러 대화, 모델을 지웠을 때 대부분의 애플리케이션에서 아티팩트가 디스크에 남아있어 복구, 파싱 가능
 - 플랫폼 : LM Studio (버전 0.3.14), Msty (버전 1.8.5), Jan (버전 0.5.16), GPT4All (버전 3.10.0)
 - GPT4ALL의 경우 앱 내에서 삭제한 이후에 아티팩트 복구가 전혀 불가능함.

c. 링크

- <https://www.sciencedirect.com/science/article/pii/S2666281725001271>
- LangurTrace
 - <https://github.com/jeongramon/LangurTrace>

3. ForensicLLM: A local large language model for digital forensics - ForensicLLM: 디지털 포렌식을 위한 로컬 대규모 언어 모델 (Binaya Sharma, James Ghawaly, Kyle McCleary, Andrew M. Webb, Ibrahim Baggili)

a. 연구 배경

- 디지털 포렌식 분야에 특화된 로컬 대규모 언어 모델인 ForensicLLM 개발 및 평가

b. 연구 결과

- ForensicLLM 도구 개발
 - 기존의 클라우드 기반 모델이 민감한 수사데이터를 외부로 유출할 위험이 있고 포렌식 전문 지식이 부족해 개발한 오픈소스 모델

c. 링크

- <https://www.sciencedirect.com/science/article/pii/S2666281725000113>
- Forensic-LLM
 - <https://github.com/vipulsparmar/Forensic-LLM>

4. 디지털 포렌식 표준(아티팩트 정의)

a. 링크

- <https://artifacts.readthedocs.io/en/latest/>